

Name of the Student	K.Prabhakar Gowd
Reg. No	192425335
GitHub Link	https://github.com/kprabhakargowd4-gif/Deep-learning-.git

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning

COURSE CODE: MLA0409

COURSE OUTCOME COVERED

CO 2:Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)

CO Weightage : 15%

Assessment Tool Weightage: 40%

Duration: 90 minutes

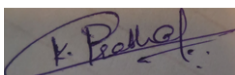
Marks: 25

Code of Conduct:

I __K.Prabhakar Gowd / 192425335__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student: K.Prabhakar Gowd

Scenario-Based Questions CO-AT2

QUESTIONS: 05

Total Marks:25

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

2. Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

3. Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

1. Unsupervised Training of Neural Networks, Auto Encoders

Scenario: A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records.

Answer:

The most suitable deep learning technique is an **Autoencoder-based unsupervised learning model**. Since the ECG records are unlabeled, an autoencoder can learn the normal patterns and identify abnormal ECG signals based on reconstruction error.

Why Autoencoder?

An autoencoder learns a compressed representation of input data without requiring labeled examples. It consists of three main components:

Input ECG → Encoder → Latent Representation → Decoder → Reconstructed ECG

The model is trained to minimize the difference between the original ECG signal and its reconstructed version.

Workflow:

1. **ECG Data Collection:** Thousands of ECG records are collected without labels.
2. **Preprocessing:** Remove noise, normalize signal values, and divide ECG recordings into suitable time windows.
3. **Encoder:** The encoder converts the high-dimensional ECG signal into a lower-dimensional latent representation.
4. **Latent Space:** Important ECG characteristics such as waveform patterns, heart rate variations, and signal structures are represented.
5. **Decoder:** The decoder reconstructs the original ECG signal from the latent representation.
6. **Training:** The autoencoder learns to reconstruct common/normal ECG patterns.
7. **Anomaly Detection:** When an abnormal ECG is given, it generally produces a higher reconstruction error because its pattern differs from what the model learned.

8. **Decision:** If reconstruction error exceeds a predefined threshold, the ECG can be flagged as potentially abnormal.

Concept Linkage:

Unlabeled ECG Data → Unsupervised Learning → Autoencoder → Representation Learning → Reconstruction Error → Anomaly Detection

Conclusion:

Therefore, an **autoencoder** is **highly suitable for unsupervised ECG anomaly detection** because it does not require manually labeled abnormal records and can automatically learn useful ECG representations. It can be used as an initial screening system, although medical diagnosis should still involve clinical validation.

2. Auto Encoders, Representation Learning

Scenario: A retail company wants to reduce the dimensionality of customer purchase data before customer segmentation.

Answer:

An **autoencoder** can be used for dimensionality reduction by learning a compact representation of customer purchase behavior while preserving the most important information.

For example, customer data may contain hundreds of features such as:

- Product categories purchased
- Purchase frequency
- Spending amount
- Discount usage
- Online/offline purchases
- Brand preferences
- Seasonal purchases

Using all these features directly for clustering can make segmentation more complex. An autoencoder can transform these high-dimensional features into a smaller latent representation.

Architecture:

Customer Purchase Data → Encoder → Latent Representation → Decoder → Reconstructed Data

Workflow:

1. **Collect Data:** Gather customer purchase records.
2. **Preprocess:** Handle missing values and normalize numerical features.
3. **Encoder:** The encoder gradually reduces the number of features.
4. **Bottleneck Layer:** A small latent layer stores the most important characteristics of customer behavior.
5. **Decoder:** The decoder attempts to reconstruct the original purchase data.
6. **Training:** The model minimizes reconstruction loss between the original and reconstructed data.
7. **Feature Extraction:** After training, the decoder can be removed and the bottleneck representation can be used as a compact feature set.
8. **Customer Segmentation:** Apply clustering algorithms such as **K-Means** to the latent representations.

Example:

Suppose the original dataset contains **100 customer features**.

100 features → 50 → 20 → 5 latent features

The 5 latent features may capture patterns such as:

- High-value customer behavior
- Frequent purchasing
- Discount sensitivity
- Product preference

- Seasonal buying behavior

Concept Linkage:

High-Dimensional Purchase Data → Encoder → Low-Dimensional Latent Space → Representation Learning → K-Means → Customer Segments

Advantages:

- Reduces dimensionality
- Removes redundant information
- Learns nonlinear relationships
- Produces useful representations for clustering
- Can improve visualization and computational efficiency

Conclusion:

Thus, an autoencoder provides an effective **representation-learning-based dimensionality reduction technique**. The learned latent features can then be supplied to a clustering algorithm to produce meaningful customer segments.

3. Width and Depth of Neural Networks, Activation Functions

Scenario: A facial recognition system performs poorly when trained using a shallow neural network.

Answer:

A shallow neural network may not have sufficient capacity to learn the complex hierarchical features required for facial recognition. Therefore, the system should be improved by increasing **network depth**, selecting an appropriate **width**, and using effective **nonlinear activation functions**.

Recommended Architecture:

Instead of:

Input → One Hidden Layer → Output

use a deeper architecture:

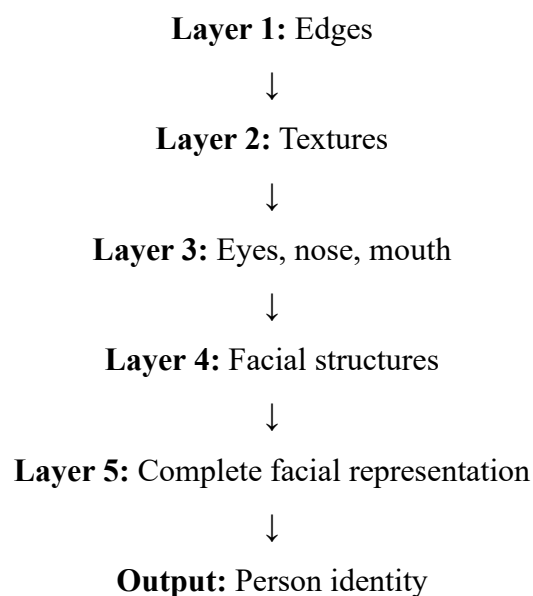
Input Image → Feature Layers → Deeper Hidden Layers → High-Level Features → Output

For image-based facial recognition, a **Convolutional Neural Network (CNN)** is particularly appropriate.

1. Increase Depth

Adding more hidden layers allows the network to learn hierarchical features.

For example:



Thus, depth enables progressive **representation learning**.

2. Increase Width Carefully

Width refers to the number of neurons or filters in a layer.

A wider network can learn more features, but excessively increasing width may:

- Increase computational cost
- Increase memory requirements
- Cause overfitting

Therefore, the width should be increased gradually and controlled using regularization.

3. Activation Functions

Nonlinear activation functions allow deep networks to learn complex relationships.

ReLU is generally suitable for hidden layers:

It is computationally efficient and helps reduce the vanishing-gradient problem compared with traditional sigmoid/tanh activations in many deep networks.

For the final classification layer, **Softmax** can be used when identifying one person among multiple known classes.

4. Additional Improvements

The architecture can also use:

- **Batch Normalization** for stable training
- **Dropout** to reduce overfitting
- **Pooling layers** to reduce spatial dimensions
- **Data augmentation** to improve robustness
- **Transfer learning** using pretrained CNN models

Concept Linkage:

Shallow Network → Limited Feature Learning → Increase Depth + Appropriate Width → Nonlinear Activations → Hierarchical Representation Learning → Better Facial Recognition

Conclusion:

The facial recognition system should use a deeper neural architecture, preferably a CNN, with appropriate width and nonlinear activation functions such as ReLU. This enables the model to learn increasingly complex facial representations and significantly improve recognition performance.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training

Scenario: A streaming service wants to recommend movies based on users' viewing history without labeled data.

Answer:

A **Restricted Boltzmann Machine (RBM)** can be used as an unsupervised learning model to discover hidden patterns in users' movie preferences and generate personalized recommendations.

An RBM consists of two layers:

- **Visible layer:** Represents observed data such as movies watched by a user.
- **Hidden layer:** Learns latent features representing user preferences.

There are **no connections between neurons within the same layer**. Connections exist only between visible and hidden layers.

Architecture:

User Movie History → Visible Layer ↔ Hidden Layer → Learned Preferences → Movie Recommendation

Workflow:

1. Create User-Movie Matrix

For example:

User	Movie A	Movie B	Movie C	Movie D
User 1	1	1	0	0
User 2	1	0	1	0
User 3	0	1	0	1

Here, 1 indicates that a user watched or liked a movie, while 0 indicates that the movie was not watched/observed.

2. **Visible Layer**

The user's movie history is provided to the visible neurons.

3. **Hidden Layer**

The RBM learns hidden patterns such as:

- Action preference
- Comedy preference
- Sci-fi preference
- Romance preference

These are **latent representations** rather than manually assigned labels.

4. **Unsupervised Training**

The RBM learns by adjusting weights so that it can reconstruct the observed user-movie interactions.

5. **Recommendation**

For a user, the trained RBM can estimate the probability that the user would interact with movies they have not previously watched.

6. **Rank Movies**

Movies with the highest predicted preference scores can be recommended to the user.

Concept Linkage:

Viewing History → Visible Layer → Hidden Features → Unsupervised Representation Learning → Preference Prediction → Personalized Recommendations

Advantages:

- Does not require manually labeled training data
- Learns hidden user preferences
- Captures relationships between users and movies
- Can provide personalized recommendations

Conclusion:

Therefore, an RBM can transform raw viewing history into meaningful latent preference representations and use them to estimate unseen movie preferences, making it suitable for a personalized recommendation engine.

5. Deep Learning Applications, Representation Learning

Scenario: A manufacturing company wants to detect defective products from sensor readings collected every second.

Answer:

A deep learning system based on **representation learning and neural networks** can automatically learn patterns from continuous sensor data and classify products as **defective or non-defective**.

Sensors may measure:

- Temperature
- Pressure
- Vibration
- Speed
- Humidity
- Voltage
- Machine noise

Since measurements are collected every second, the data forms a **time series**.

Proposed Architecture:

Sensor Data → Preprocessing → Feature Representation → Deep Neural Network → Classification → Defect/Normal

A suitable architecture could combine **1D CNN and LSTM/GRU**, depending on the characteristics of the sensor data.

Workflow:

1. Data Collection

Collect timestamped sensor readings from manufacturing machines.

Example:

Time	Temperature	Pressure	Vibration	Status
1 sec	72	40	0.2	Normal
2 sec	73	41	0.2	Normal
3 sec	91	58	1.8	Defective

2. Preprocessing

- Handle missing values
- Remove obvious noise
- Normalize sensor values
- Divide continuous readings into time windows

For example:

60 seconds of readings → One input sequence

3. Representation Learning

Instead of manually designing features, the neural network learns useful representations automatically.

A **1D CNN** can learn local patterns such as sudden changes in temperature, pressure, or vibration.

An **LSTM/GRU** can learn temporal relationships, such as:

Normal → Increasing vibration → Increasing temperature → Pressure spike → Defect

4. Classification

The learned representation is passed to fully connected layers.

For binary classification:

Output = Normal / Defective

A sigmoid activation can produce a probability of defect.

5. Training

The model is trained using historical sensor sequences and their defect status.

A suitable loss function for binary classification is **binary cross-entropy**.

6. Real-Time Detection

When new sensor readings arrive:

New Sensor Data → Trained Model → Defect Probability → Alert

If the probability exceeds a predefined threshold, the system can generate a warning for the manufacturing team.

Concept Linkage:

**Continuous Sensor Data → Time-Series Representation → CNN/LSTM Feature Learning
→ Neural Network Classification → Defect Detection → Real-Time Alert**

Advantages:

- Automatically learns useful features
- Handles large-scale sensor data
- Detects complex nonlinear patterns
- Can support real-time monitoring
- Reduces dependence on manually engineered features
- Can potentially identify early warning patterns before complete product failure

Conclusion:

A deep learning model using **representation learning with CNN/LSTM or GRU** can effectively learn spatial/local and temporal patterns from sensor readings. The resulting representations can be used to classify products as normal or defective and support real-time quality control.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand